4 Лабораторная работа № 4 «Построение синтаксического анализатора»

Основная задача синтаксического анализа - проверка исходной программы на соответствие грамматике языка программирования. Следует еще раз напомнить, что синтаксический анализ производится над кодом программы, который получен на выходе лексического анализа. Результат синтаксического анализа, который часто называется разбором, представляется в виде дерева разбора. Данное дерево должно демонстрировать вывод исходной программы как цепочки символов из начального символа грамматики.

Для построения дерева разбора используются различные методы синтаксического анализа, каждый из которых имеет свои особенности. В нашем случае будет рассмотрен пример разбора «сверху-вниз», который называется рекурсивным спуском. Основная особенность рекурсивного спуска заключается в том, что для анализа каждого нетерминала используется отдельная семантическая процедура, а многократное обращение к ней в процессе анализа и дало в название методу – «рекурсивный спуск».

Рассмотрим суть синтаксического анализа методом рекурсивного спуска на примере.

Вначале построим грамматику языка программирования, описанного в лабораторной работе № 1. Его грамматика, начальным символом которой является нетерминал <программа>, имеет следующий вид:

<программа> ::= *PROC MAIN;*<текст> *END*.

<текст> ::= (<строка>|<процедура>){<строка>|<процедура>}

<строка> ::= <описание>

|<отд.оператор>

<описание> ::= *DCL*<идентфикатор>{,<идентификатор>}*DEC FIXED;*

<отд.оператор> ::= <идентификатор>:<оператор>

|<оператор>

<оператор> ::= <идентификатор>:=<выражение>

| *GOTO* <идентификатор>

| <условный оператор>

<процедура> ::= *PROC* <идентификатор>;<текст>*END*;

<условный оператор> ::= *IF* <условие> *THEN* <оператор>

<условие> ::= <выражение><неравенство><выражение>

<выражение> ::= <терм>{+<терм>}

<терм> ::= <множитель>{*<множитель>}

<множитель> ::= <аргумент>|(<выражение>)

<аргумент> ::= <идентификатор>

|<константа>

<идентификатор> ::= <буква>{<буква>|<цифра>}

<константа> ::= <число>

|.<число>

|<число>.<число>

|<число>.

<буква> ::= *A* | *B* |...| *Y* | *Z*

<цифра> ::= *0* | *1* |...| *8* | *9*

Теперь введем некоторые термины.

Цель. Под ней будем понимать некоторый нетерминальный символ, который в данный момент времени является для дерева концевым. Задача заключается в том, чтобы из него продолжить построение дерева (найти вывод). И так далее до тех пор, пока в качестве цели не окажется терминальный символ.

Построение дерева разбора в методах синтаксического анализа «сверху-вниз» всегда производится слева направо (в качестве цели всегда выбирается самый левый концевой символ). Здесь очень важно найти в

разбираемой цепочке входной программы самую левую подцепочку, выводимую из цели. Эта подцепочка называется фразой для нетерминала U и в методах синтаксического анализа играет ключевую роль. Для ясности дадим формальное определение понятия фразы.

Фраза. Пусть задана грамматика G[Z], где Z – начальный символ и некоторая цепочка xUy, тогда цепочка *u* называется фразой для нетерминала U, если выполняется два условия:

$$Z \quad + xUy$$

$$U \quad + u$$

Если вывод U *u* является простым, то и фраза *u* также называется простой.

Итак, при разборе семантическая процедура нетерминального символа (цели) всегда для разбираемой цели ищет самую левую простую фразу, заменяет ее на цель и далее разбор повторяется для новой цели.

Для устранения лишних возвратов в рекурсивном спуске в качестве контекста используется единственный символ, следующий за разобранной частью фразы, который постоянно отслеживается.

Рекурсивная процедура для нетерминала U ищет фразу следующим образом: сравнивает разбираемый символ с правыми частями правил для данного нетерминала U, выбирает подходящее правило грамматики и по мере необходимости вызывает другие процедуры для используемых в правиле нетерминалов. Таким образом происходит спуск до уровня правил, не имеющих в правых частях нетерминалов.

Процедура, удачно завершившая разбор соответствующего ей правила, завершает свою работу, возвращая в точку вызова признак успешного окончания. В противном случае (если соответствующее ей правило (правила) неприменимы в данном контексте), в точку вызова возвращается признак неудачи, и вызывающая процедура переходит к разбору по следующему правилу, если таковое имеется, или, в свою очередь, при его отсутствии возвращает признак неудачи.

При программной реализации данного алгоритма были определены следующие переменные и процедуры, за которыми закреплены определенные функциональные назначения.

1. Переменная NXTSYMB - глобальная переменная, содержащая символ, следующий за разбираемым символом. NXTSYMB содержит символ (лексему) исходной программы, который будет обрабатываться следующим, а при поиске новой цели в переменной NXTSYMB всегда находится первый символ, который должен быть исследован.

При выходе из процедуры с сообщением об успехе символ, следующий за закрытой подцепочкой, помещается в NXTSYMB.

2. Процедура SCAN готовит очередной символ исходной программы и помещает его в NXTSYMB.

3. Процедура ERROR предназначена для обработки ошибочных ситуаций.

Ниже в таблице 4.1 собраны нетерминалы грамматики и имена соответствующих им рекурсивных семантических процедур.

Таблица 4.1

Нетерминальные символы грамматики	Имена рекурсивных процедур
<программа>	PROGRAM
<текст>	TEXT
<строка>	LINE
<описание>	DECLARE
<отд.оператор>	SEP_OPER
<оператор>	OPERATOR
<процедура>	PROCED
<условный оператор>	IF_OPER
<условие>	CONDITION
<выражение>	EXPRESSION
<терм>	TERM
<множитель>	FACTOR
<аргумент>	ARGUMENT
<идентификатор>	IDENT
<константа>	CONST

Процедуры IDENT и CONST в отличие от всех остальных процедур возвращают значение ИСТИНА или ЛОЖЬ в зависимости от того, является ли лексема, содержащаяся в переменной NXTSYMB, идентификатором или константой соответственно.

При инициализации синтаксического анализатора идет обращение к процедуре SCAN, которая помещает первый символ исходной программы в глобальную переменную NXTSYMB и вызывает процедуру PROGRAM.

Для рассмотренной грамматики программа синтаксического анализатора по методу рекурсивного спуска приведена ниже.

```
procedure PROGRAM;
begin
    SCAN;
    if NXTSYMB<>'PROC' then ERROR;
    SCAN;
    if NXTSYMB<>'MAIN' then ERROR;
    if NXTSYMB<>';' then ERROR;
    SCAN;
    TEXT;
    SCAN;
    if NXTSYMB<>'END' then ERROR;
    SCAN;
    if NXTSYMB<>'.' then ERROR;
end.

procedure TEXT;
while NXTSYMB<>'END' do
    if NXTSYMB='PROC' then PROCED
        else LINE;
end;

procedure PROCED;
begin
    SCAN;
    if NOT IDENT (NXTSYMB) then ERROR;
    SCAN;
    if NXTSYMB<>';' then ERROR;
    SCAN;
```

```
    while NXTSYMB<>'END' do begin
        LINE;
        SCAN;
    end;
    SCAN;
    if NXTSYMB<>';' then ERROR;
end;

procedure LINE;
if NXTSYMB='DCL' then DECLARE
    else SEP_OPER;
end;

procedure DECLARE;
begin
    SCAN;
    if NOT IDENT (NXTSYMB) then ERROR;
    SCAN;
    while NXTSYMB=',' do begin
        SCAN;
        if NOT IDENT (NXTSYMB) then ERROR;
        SCAN;
    end;
    if NXTSYMB<>'DEC' then ERROR;
    SCAN;
    if NXTSYMB<>'FIXED' then ERROR;
    SCAN;
    if NXTSYMB<>';' then ERROR;
    SCAN;
end;
```

```
procedure SEP_OPER;
if NXTSYMB='IF' OR NXTSYMB='GOTO' then OPERATOR
    else begin
        if NOT IDENT (NXTSYMB) then ERROR;
        SCAN;
        if NXTSYMB=':' then begin
                SCAN;
                OPERATOR;
        end;
        else if NXTSYMB=':=' then begin
                    SCAN;
                    EXPRESSION;
                end;
            else
                ERROR;
end;

procedure OPERATOR;
if NXTSYMB='IF' then IF_OPER
    else if NXTSYMB='GOTO' then begin
        SCAN;
        if NOT IDENT (NXTSYMB) then ERROR;
    end;
else begin
    IDENTIFIER;
    if NXTSYMB<>':=' then ERROR
        else begin
            SCAN;
            EXPRESSION;
```

```
        end;
    end;
end;

procedure IF_OPER;
begin
    SCAN;
    CONDITION;
    if NXTSYMB<>'THEN' then ERROR
        else begin
            SCAN;
            OPERATOR;
        end;
end;

procedure CONDITION;
begin
    EXPRESSION;
    if NXTSYMB<>'>' then
        if NXTSYMB<>'<' then ERROR;
    SCAN;
    EXPRESSION;
end;

procedure EXPRESSION;
begin
    TERM;
    SCAN;
    while NXTSYMB='+' do begin
        SCAN;
```

```
        FACTOR;
    end;
end;

procedure TERM;
begin
    FACTOR;
    while NXTSYMB='*' do begin
        SCAN;
        FACTOR;
    end;
end;

procedure FACTOR;
if NXTSYMB='(' then begin
        SCAN;
    EXPRESSION;
        if NXTSYMB<>')' then ERROR;
            else
                SCAN;
        end;
    else begin
            ARGUMENT;
            SCAN;
        end;
end;

procedure ARGUMENT;
if NOT CONST(NXTSYMB) then
```

```
            if NOT IDENT(NXTSYMB)) then ERROR;
        end;
```

При выполнении лабораторной работы № 4 студенты должны внимательно ознакомиться с предлагаемым подходом и по аналогии с ним выполнить разработку синтаксического анализатора языка программирования, предложенного в варианте задания к лабораторной работе.